You *can* manipulate a tuple with functions, but you really shouldn't. The reason is that they are fixed-size containers and operations on them are more costly.

When you add to the beginning of a list, it's a quick operation that touches a minimal amount of memory. When you add to a tuple, it completely recreates the data structure in memory.

Tuples are often used to represent the success or failure of a function or as containers for functions with multiple return values. It's easiest to see this in a real Elixir function:

```
$ iex
iex> File.read("./")
{:error, :eisdir}

iex> File.ls("./")
{:ok, ["your", "files"]}

iex> File.read!("./")
** (File.Error) could not read file ".": illegal operation on a directory
```

Similar to Ruby, the convention is to put a ! symbol at the end of a function if it raises an error. This is why there are File.read!/1 and File.ls!/1 functions that raise errors instead of returning a tuple result.

Tuple results play well with pattern matching, so you can easily handle the success or failure of an operation without catching errors. This is considered pragmatic Elixir and is commonly seen. We'll cover that in the next chapter.

Performance Dangers

Lists look similar to Ruby arrays, so it's easy to think that you can use them the same way. But there's a nonobvious risk that you may run into. Elixir lists are linked lists with O(n) performance on many common operations.

This section needs a disclaimer. You'll likely not run into problems in your everyday usage of Elixir lists due to how large a list would need to be to give you problems. However, it's still an important topic to understand when learning Elixir, or if you're implementing algorithms in Elixir. Elixirists use lists a lot, so they're quite useful despite any performance dangers.

When you access an element in an Elixir list, the entire list up to that element is traversed. As you perform operations on a list that's large, the CPU requirements and time needed to complete the operation increase.

List performance is why it's not common to access list elements by index in Elixir. It's common in Ruby to access a list element using a list[5] syntax or to